In [1]:
```
import pandas as pd

df = pd.read_csv("../apple/Desktop/healthcare_real_time_dataset.csv")

df.head()
```

Out[1]:

	Age	Gender	BMI	Smoking Status	Alcohol Consumption (per week)	Physical Activity (hours/week)	Sleep Duration (hours/day)	Chronic Disease History	Stress Level (1-10)	Health Risk Level
0	56	Other	23.15	Never	1	0.1	4.9	Heart Disease	5	High
1	69	Male	20.83	Former	3	9.8	7.2	NaN	8	Moderate
2	46	Female	21.55	Never	0	9.7	5.4	Hypertension	7	Low
3	32	Other	27.47	Former	18	4.0	4.8	Hypertension	8	Low
4	60	Male	27.09	Current	9	7.3	5.9	Diabetes	8	High

In [2]:
```
df.info()
print("\nMissing Values:\n")
print(df.isnull().sum())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 10 columns):
 #   Column                          Non-Null Count  Dtype
---  ------                          --------------  -----
 0   Age                             1000 non-null   int64
 1   Gender                          1000 non-null   object
 2   BMI                             1000 non-null   float64
 3   Smoking Status                  1000 non-null   object
 4   Alcohol Consumption (per week)  1000 non-null   int64
 5   Physical Activity (hours/week)  1000 non-null   float64
 6   Sleep Duration (hours/day)      1000 non-null   float64
 7   Chronic Disease History         765 non-null    object
 8   Stress Level (1-10)             1000 non-null   int64
 9   Health Risk Level               1000 non-null   object
dtypes: float64(3), int64(3), object(4)
memory usage: 78.2+ KB

Missing Values:

Age                                 0
Gender                              0
BMI                                 0
Smoking Status                      0
Alcohol Consumption (per week)      0
Physical Activity (hours/week)      0
Sleep Duration (hours/day)          0
Chronic Disease History           235
Stress Level (1-10)                 0
Health Risk Level                   0
dtype: int64
```

In [3]:

```
df["Chronic Disease History"] = df["Chronic Disease History"].fillna("No Disease")

# Verify
print(df.isnull().sum())
```

```
Age                                0
Gender                             0
BMI                                0
Smoking Status                     0
Alcohol Consumption (per week)     0
Physical Activity (hours/week)     0
Sleep Duration (hours/day)         0
Chronic Disease History            0
Stress Level (1-10)                0
Health Risk Level                  0
dtype: int64
```

In [4]:

```
from sklearn.preprocessing import LabelEncoder

# Encode target variable
le = LabelEncoder()
df["Health Risk Level"] = le.fit_transform(df["Health Risk Level"])

# One-hot encode categorical features
df = pd.get_dummies(df, columns=["Gender", "Smoking Status", "Chronic Disease History"])

df.head()
```

Out[4]:

	Age	BMI	Alcohol Consumption (per week)	Physical Activity (hours/week)	Sleep Duration (hours/day)	Stress Level (1-10)	Health Risk Level	Gender_Female	Gender_Male	Gender_Other	Smok Status_Curr
0	56	23.15	1	0.1	4.9	5	0	False	False	True	F
1	69	20.83	3	9.8	7.2	8	2	False	True	False	F
2	46	21.55	0	9.7	5.4	7	1	True	False	False	F
3	32	27.47	18	4.0	4.8	8	1	False	False	True	F
4	60	27.09	9	7.3	5.9	8	0	False	True	False	T

In [5]:

```
X = df.drop("Health Risk Level", axis=1)
y = df["Health Risk Level"]
```

```
print("Feature shape:", X.shape)
print("Target shape:", y.shape)
```

```
Feature shape: (1000, 16)
Target shape: (1000,)
```

In [6]:
```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Scale
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

print("Training size:", X_train.shape)
print("Testing size:", X_test.shape)
```

```
Training size: (800, 16)
Testing size: (200, 16)
```

In [7]:
```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=1000)

model.fit(X_train, y_train)

print("Model training complete.")
```

```
Model training complete.
```

In [8]:
```
from sklearn.metrics import accuracy_score, classification_report

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)
```

```
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))
```

```
Accuracy: 0.34

Classification Report:

              precision    recall  f1-score   support

           0       0.29      0.30      0.30        69
           1       0.39      0.48      0.43        73
           2       0.31      0.21      0.25        58

    accuracy                           0.34       200
   macro avg       0.33      0.33      0.33       200
weighted avg       0.33      0.34      0.33       200
```

In [9]:

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(n_estimators=200, random_state=42)

rf.fit(X_train, y_train)

y_pred_rf = rf.predict(X_test)

from sklearn.metrics import accuracy_score, classification_report

print("Random Forest Accuracy:", accuracy_score(y_test, y_pred_rf))
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred_rf))
```

```
Random Forest Accuracy: 0.275

Classification Report:

              precision    recall  f1-score   support

           0       0.19      0.14      0.17        69
           1       0.37      0.48      0.42        73
           2       0.19      0.17      0.18        58

    accuracy                           0.28       200
   macro avg       0.25      0.27      0.25       200
weighted avg       0.26      0.28      0.26       200
```

In [10]:
```
print(le.classes_)
```

```
['High' 'Low' 'Moderate']
```

In [11]:
```
def give_recommendation(predicted_label):

    if predicted_label == 0:  # High Risk
        return """
        HIGH RISK:
        - Immediate medical consultation recommended
        - Reduce alcohol and smoking
        - Strict diet control
        - Regular monitoring of vitals
        - Stress management required
        """

    elif predicted_label == 2:  # Moderate Risk
        return """
        MODERATE RISK:
        - Improve physical activity
        - Maintain balanced diet
        - Reduce stress levels
        - Routine health checkup every 6 months
        """

    else:  # Low Risk
```

```
        return """
        LOW RISK:
        - Maintain healthy lifestyle
        - Regular exercise
        - Balanced nutrition
        - Annual health checkup
        """
```

In [12]:
```
sample_prediction = rf.predict([X_test[0]])[0]

print("Predicted Class:", sample_prediction)
print(give_recommendation(sample_prediction))
```

```
Predicted Class: 1

        LOW RISK:
        - Maintain healthy lifestyle
        - Regular exercise
        - Balanced nutrition
        - Annual health checkup
```

In [ ]: